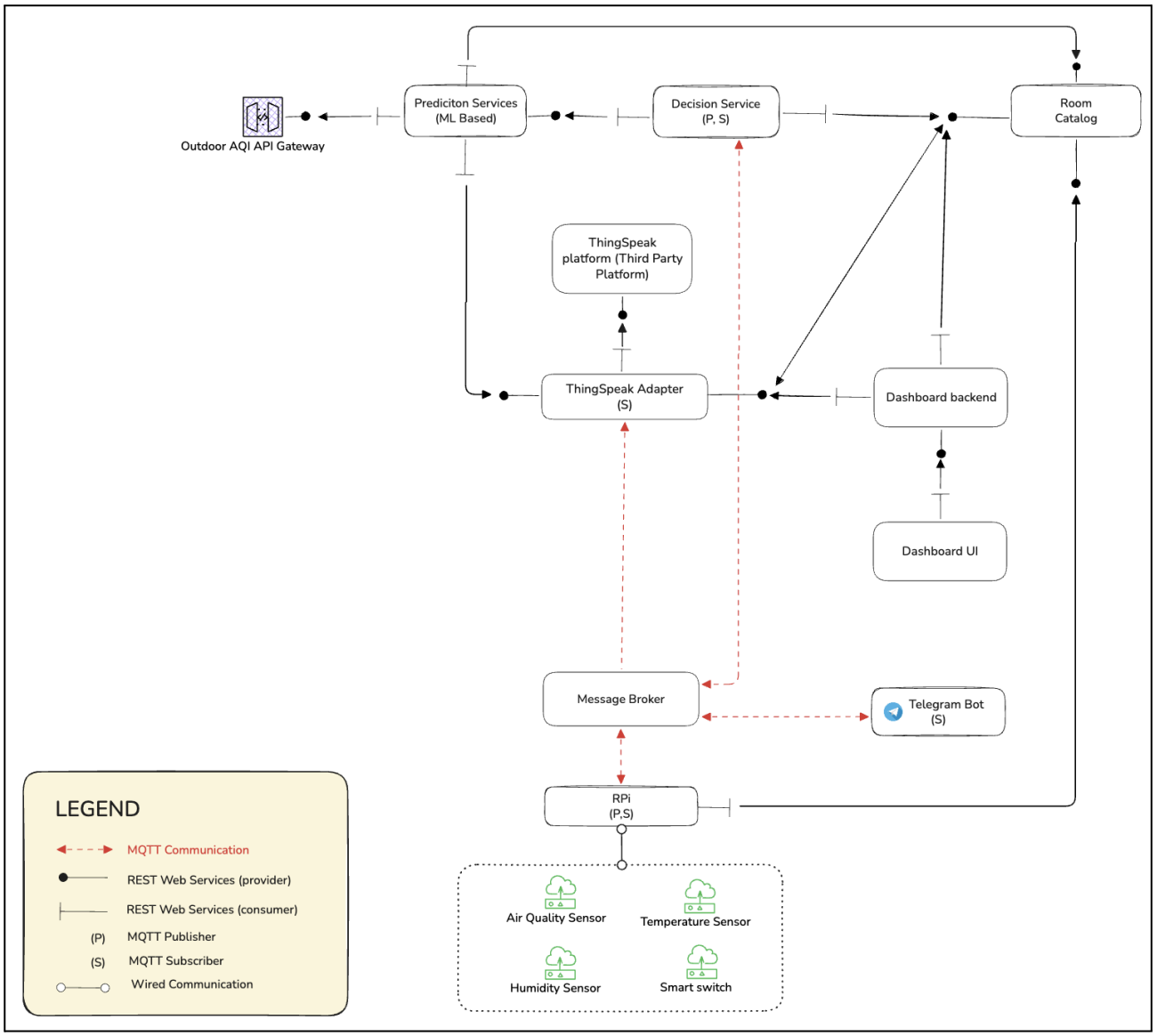
1. Name of Use Cases

Name of the Use Case	AirGuard: A Scalable IoT Platform for IAQ Monitoring and Adaptive Ventilation Control
Version No.	v0.4
Submission Date	25th Mar. 2026
Team Members (with student ids)	**Yang Ming: s337032** **Mateus Lanzoni Trejos: s350240** **Syed Umer Farooq Shah: s359420** **Li Ruolin: s337356**

2. Scope and Objectives of Function

Scope and Objectives of Use Case	
Scope	The proposed IoT platform provides real-time monitoring, data analytics, and adaptive control services for smart Air Quality management in residential and commercial buildings
Objective(s)	1. Provide an IAQ monitoring and adaptive ventilation control solution for smart homes and smart buildings. 2. Integrate outdoor APIs and indoor sensor data to automatically maintain healthy IAQ levels using adaptive control. 3. Develop a scalable system that allow users to manage devices and configure IAQ goals and receive notifications.
Domain(s)	Environmental Monitoring Systems, Smart Buildings
Stakeholder(s)	Building Managers, Residents, Kitchen Managers, System Administrators, Facility Managers
Short description	The system automates indoor air quality management by using a cloud-based microservice architecture and a Raspberry-Pi-based device connector. It continuously collects outdoor API data and indoor sensor measurements, performs real-time analytics, and triggers ventilation or purification actions based on user-defined IAQ goals. The main features are: 1. Manage and control smart IAQ devices. 2. Automate system real-time actions based on user settings of IAQ level or rule-based strategies. 3. Analyze the historical IAQ data to arrange the future actions, such as when to activate devices to exchange air. 4. Provide a web dashboard for monitoring, visualization and user configuration. 5. Send alerts based on IAQ conditions and system executed actions.

3. Diagram of Use Case



4. Complete description of the system

4.1 Environmental Sensing & Device Connector Layer

The Environmental Sensing & Device Connector Layer is responsible for acquiring real-time indoor environmental data and executing physical actuation commands.

Raspberry Pi Device Connector

The Raspberry Pi acts as an edge gateway between the physical sensors and the cloud backend. It performs basic validation of sensor measurements,and timestamping, and publishes environmental data to the Message Broker using the MQTT protocol. The Raspberry Pi also subscribes to MQTT topics to receive actuation commands and forwards them to the connected actuators for execution.

This layered approach separates low-level hardware control from higher-level communication and processing, improving system reliability and scalability.

Key responsibilities:

- Continuous acquisition of temperature, humidity, CO_2, PM2.5, and VOC readings.
- Local validation and preprocessing of raw sensor data.
- Publishing environmental data to the backend via MQTT message pipelines.
- Receiving device actuation commands (fan/AC ON/OFF, airflow adjustments) via MQTT. and publish success messages as feedback.

4.2 Message Communication Layer

4.2.1 The Message Communication Overview

It provides asynchronous communication between device connectors and backend microservices. It is implemented using an MQTT message broker, which enables lightweight, low-latency, publish/subscribe data exchange suitable for continuous sensor data streams.

Sensor measurements published by the Raspberry Pi Device Connector are transmitted through MQTT topics to backend services. Actuation commands issued by the Decision Service are similarly published to MQTT topics and consumed by the Raspberry Pi Device Connector for physical execution.

This communication model decouples data producers from consumers and allows the system to scale across multiple rooms and buildings without direct point-to-point connections.

RESTful HTTP APIs are strategically employed to handle synchronous, request-response interactions, complementing the real-time MQTT data plane.

Specifically, REST is utilized for external integrations, such as fetching third-party Outdoor AQI via the API Gateway, and for metadata management, allowing the Decision Service to synchronously query the Room Catalog for actuation strategies. Furthermore, REST APIs facilitate historical data operations, enabling the system to push aggregated telemetry to the ThingSpeak platform, retrieve

long-term datasets for the Prediction Service's machine learning models, and serve analytics to the Web Dashboard for user visualization and configuration updates.

Key responsibilities:

- Decoupling Raspberry Pi communication from cloud microservices.
- Routing sensor readings to analytics and decision-making services.
- Delivering actuation commands back to the Raspberry Pi.
- Ensuring reliable real-time communication even under high deployment loads.
- Historical Storage & Dashboard Integration
- Synchronous context and configuration

4.2.2 Message Broker

Description: The Message Broker is the central nervous system of the platform, enabling asynchronous, decoupled communication between edge devices and cloud microservices.

Inputs:

- Device registration payloads, heartbeat and actuation status messages published by sensors via MQTT
- Messages published by the Decision Service via MQTT, i.e. alerts, action commands
- Control from dashboard backend via MQTT
- RPI sensing data and success action messages via MQTT

Processing Logic:

- Topic Routing: It matches incoming messages to subscribers based on topic hierarchies (e.g., routing `messages` to the Analysis Service).

Outputs:

- Push messages delivered to subscribed clients (Decision Service, Raspberry Pi Device Connector, Telegram Bot, and ThingSpeak Adapter) via MQTT.

4.3 Analytics & Decision-Making Layer (Backend)

The Analytics & Decision-Making Layer consists of a set of independent microservices responsible for processing sensor data, generating predictions, and controlling actuation behavior. Each service exposes REST interfaces and communicates internally using MQTT where appropriate.

4.3.1 Room Catalog

Description: The **Room Catalog** acts as the central registry and configuration service of the system. It functions as both a **Device Registry** and a **Service Registry**, maintaining metadata about all connected devices, services, and room configurations.

Every device and backend service must **register itself in the Catalog when it starts**, allowing other components to discover available resources and obtain configuration information.

The Catalog also stores **user preferences, device status, and room configurations**, which are used by the Decision Service to determine appropriate actuation strategies.

Inputs

- Device registration requests from Raspberry Pi Device Connectors via REST API during startup.
- Service registration requests from backend services (Prediction Service, Decision Service, Dashboard Backend, ThingSpeak Adapter) via REST API.
- Service registration request sent to the Room Catalog via REST API during service startup.
- Device status updates sent by devices via REST API (PUT).
- User preference updates from Dashboard Backend via REST API.
- Device and service discovery requests from Decision Service and Dashboard Backend via REST API.

Processing Logic

- Services register itself with the Room Catalog at startup to enable service discovery.
- **Device Registration:** When a registration request is received via REST, the Room Catalog checks whether the device ID already exists. If it does not, a new device record is created storing the device ID, type, and so on. If the device ID already exists, the existing record is updated with the latest information.
- **Status Tracking:** Devices periodically update their status using **REST PUT requests**. Devices periodically update their operational status via REST PUT requests. Each update refreshes the device status (online, offline, error) and updates the last-seen timestamp stored in the Catalog. If no heartbeat is received from a device within a defined timeout period (e.g., 60 seconds), the device status is automatically set to offline.
- **Preference Management:** Each newly registered device is automatically assigned a set of default preferences (e.g., target temperature: 24°C, humidity range: 40–60%, schedule: 08:00–22:00). User preferences such as target IAQ levels or temperature ranges are updated through the Web Dashboard UI, which sends REST API requests to the Dashboard Backend, and are then forwarded to the Room Catalog. The Catalog validates the request and updates the stored configuration accordingly.

Outputs

- Device metadata, configuration, and current status returned to the Decision Service via REST API
- Device and service discovery information returned to backend services via REST API
- Room configuration and user preferences returned to the Dashboard Backend via REST API

4.3.2 Prediction Services (ML-Based)

Description The Prediction Service is the intelligence layer of the system. It retrieves recent and historical indoor sensor data from ThingSpeak and combines it with current outdoor AQI data to produce short-term environmental forecasts. These predictions give the Decision Service forward-looking context so it can act before conditions go out of the preferred range. The Prediction Service registers itself in the Room Catalog via REST API during startup for service discovery purposes.

Inputs:

- REST request from the Decision Service requesting the latest IAQ prediction during each decision cycle.
- Historical and latest environmental data retrieved from the ThingSpeak adaptor via REST API.
- Outdoor AQI data from the public Outdoor AQI API via REST API.

Processing Logic:

- When triggered by a REST request from the Decision Service, the Prediction Service retrieves historical and recent sensor data and combines it with the latest outdoor AQI data.
- The Prediction Services apply machine-learning or time-series forecasting techniques to estimate future IAQ evolution. By leveraging historical trends and learned patterns, the system can anticipate poor air-quality conditions and enable proactive ventilation strategies.

Outputs:

- Predicted IAQ trends returned to the Decision Service via REST API.
- Processed environmental data may be forwarded to the ThingSpeak Adapter for storage via REST API.

4.3.3 Decision Service

Description The core control logic of the system. Upon receiving a new message, it queries the Prediction Service and Room Catalog via REST to obtain the latest forecasts, device status, and user preferences, then applies rule-based or model-driven logic to determine what control actions to take. It is the only service authorized to issue control commands.

Inputs

- Prediction results from Prediction Service via REST API.
- Device status and user preferences from room catalog via REST API.
- Sensor measurement messages received from the Message Broker via MQTT (triggering the decision process).

Processing Logic:

- Reads the latest prediction from Prediction Service and queries room catalog for current device state and user preferences at each decision cycle
- Applies decision rules against user-defined thresholds.
- Checks device schedule before issuing any command; no commands are sent outside active hours
- Applies a debounce rule to avoid rapid switching (minimum 5-minute gap between state changes)
- Publishes an alert message to the Message Broker when a significant event occurs (e.g., temperature far above threshold, device offline)
- Publishes a control event record to Message Broker after each command for audit and future prediction use.

Outputs

- Control commands published to Message Broker for the Raspberry Pi Device Connector to consume via MQTT.
- Alert message published to the Message Broker via MQTT for the Telegram Bot to consume.
- Control event record written to Message Broker for the ThingSpeak Adaptor via MQTT.

4.3.4 Dashboard Backend

Description: The Dashboard Backend is an auxiliary service that prepares and aggregates data required by the **Web Dashboard user interface**. It retrieves device metadata, room configurations, and historical environmental measurements from internal services and external storage platforms. The service processes and formats this data so that the Web Dashboard can efficiently visualize environmental trends, device states, and performance metrics.

Dashboard Backend focuses on data retrieval, processing, and forwarding user configuration updates to the Room Catalog.

This separation ensures a clear distinction between user interfaces and backend processing services, as the Dashboard Backend is responsible only for data aggregation and processing, while user interaction is handled by the Web Dashboard UI and Telegram Bot independently.

Inputs

- Device metadata, room configurations, and user preferences from Room Catalog via REST API.
- Historical environmental data retrieved from the ThingSpeak Adaptor via REST API

Processing Logic:

- Retrieves historical time-series sensor data from the ThingSpeak via REST
- Processes and formats the data to generate aggregated metrics and visualization-ready datasets for the Web Dashboard

Outputs

- Processed environmental metrics and historical datasets exposed to the Web Dashboard via REST API
- Visualization-ready data used by the Web Dashboard to display historical trends, IAQ metrics, and device status.
- User preference update requests forwarded to the Room Catalog via REST API

4.3.5 Web Dashboard UI

Description: The Web Dashboard UI is the primary user-facing interface of the system, allowing users to monitor indoor air quality, visualize historical trends, and configure system preferences. It interacts with the Dashboard Backend via REST APIs to retrieve processed data and display visualization metrics.

Inputs:

- Processed environmental data and metrics retrieved from the Dashboard Backend via REST API

Processing Logic:

- Displays real-time and historical IAQ data through charts and visual components
- Provides user interaction elements for configuring system preferences and viewing device status

Outputs:

- User preference update requests and configuration changes sent to the Dashboard Backend via REST API

4.3.6 Telegram Bot

Description: The Telegram Bot acts as a lightweight user interaction interface dedicated to receiving real-time alert notifications. It subscribes directly to alert topics on the Message Broker using MQTT, ensuring that critical system events are delivered to users immediately. The bot provides a simple and responsive channel for notifying users about abnormal conditions or important system events.

Inputs

- Alert messages received from the Message Broker via MQTT

Processing Logic

- Subscribes to alert topics on the Message Broker
- Upon receiving an alert message, formats and forwards the notification to users through the Telegram platform

Outputs

- Alert notifications delivered to users through the Telegram platform.

4.3.7 ThingSpeak Adaptor and ThingSpeak (Used for Supporting Services)

Description: The ThingSpeak Adaptor serves as a bridge between the real-time MQTT architecture and the ThingSpeak cloud platform. It receives and stores all incoming sensor readings, and Decision Service control events. This data serves three purposes: providing input for ML model inference in the Prediction Service, supplying historical data for user queries handled by Dashboard Backend, and maintaining an audit trail of all control decisions made by the Decision Service.

Inputs:

- IAQ sensor data (CO2, PM2.5, Temperature, Humidity) and status subscribed from the Message Broker via MQTT.
- Control event records via MQTT broker

Processing Logic:

- The adapter buffers incoming high-frequency MQTT messages to match ThingSpeak's API rate limits.

Outputs:

- Adent as HTTP POST (REST) requests to the ThingSpeak platform for paptor aggregated data payloads sersistent storage.
- Historical and latest sensor data retrieved from ThingSpeak and returned to the Prediction Service via REST API.
- Historical sensor data retrieved from ThingSpeak and returned to the Dashboard Backend via REST API.

4.4 Actuation Layer (Fans, Extractors, HVAC/AC)

While the Decision Service makes the decisions, this Physical Actuation Layer is responsible for the mechanical execution. It consists of the physical hardware (fans, motorized windows, AC units) connected to the RPi.

For demonstration purposes, high-voltage appliances (such as AC units and air purifiers) will be emulated using smart switches to represent actuation states (ON/OFF), while their environmental impact will be simulated using synthetic data streams to close the feedback loop.

Command Flow: Unlike the cloud services, these physical devices do not process data; they simply execute actions. The workflow is as follows:

1. The Raspberry Pi subscribes to the actuation MQTT topics via the Message Broker
2. Upon receiving a command (e.g., `FAN_ON`), the Raspberry Pi relays the signal to the target device (e.g., wired sensors).
3. The target device changes its relay state to activate or deactivate the connected appliance
4. Raspberry Pi publishes a status feedback message (ON/OFF) back to the Message Broker to confirm execution via MQTT.

4.5 Scalability & Multi-Location Support

The system is built with scalability as a core principle. Multiple Raspberry Pi units can be deployed across different rooms, floors, or buildings, all connected to the same backend. Each sensing node operates independently while sharing a unified communication and analytics infrastructure. The modular microservice architecture allows new sensors, actuators, or data-processing components to be added without redesigning the system. This makes the platform suitable for small installations (single private rooms) as well as large deployments (hotels, corporate buildings, factories, commercial kitchens).

Key scalability advantages:

- Distributed sensing: multiple independent Raspberry Pi units.
- Cloud-based microservices that scale horizontally.
- Easy addition of new sensors or environmental zones.
- Suitable for small, medium, and enterprise-level deployments.

The use of MQTT-based communication and stateless microservices enables horizontal scalability across multiple rooms, buildings, and device connectors.

4.6 Data Storage Services

4.6.1 Time-Series Data Storage (ThingSpeak)

ThingSpeak is used as an external third-party platform for storing historical environmental measurements. Sensor readings and control event records are forwarded to ThingSpeak through the ThingSpeak Adapter using REST APIs.

The stored time-series data is later retrieved by the Prediction Service for machine-learning analysis and by the Dashboard Backend for visualization of historical trends and environmental metrics.

5. Desired Hardware components (only among those we can provide)

We will use synthetic or simulated data that follows realistic patterns derived from public datasets. This allows us to test the end-to-end IoT pipeline (data ingestion, analytics, control logic) without physical hardware like air quality sensors, ACs, fans.

A smart switch will be used to replace physical AC/fan controls.

Device Name	Quantity	Needed for…
Raspberry Pi	1	Running the device connector
Temperature & Humidity Sensor	1	Measuring indoor temperature and humidity
Air Quality Sensor（CO_2, PM2.5)	1	Measuring indoor air quality

Smart switch	2	Replacing physical AC/fan controls